دليل التدريب · الإصدار 0.1

تدريب وكيل flutter-agent

كيف تحوّل عملك اليومي مع Claude إلى معرفة دائمة داخل الوكيل — وكيف يخفّض ذلك استهلاك الـ tokens بدل أن يرفعه.

الحالة الراهنة: 1,664 مقطعًا مفهرسًا · 1,617 من الكود · 19 من القواعد · 27 من الأنماط · درس واحد.

الهدف من هذا الدليل: أن تصبح هذه الأرقام الثلاثة الأخيرة أكبر بكثير، وبطريقة صحيحة.

تحذير مبكر: هناك طريقة خاطئة للتدريب تُفسد الوكيل تدريجيًا. القسم 2 مخصّص لها.

المحتويات

1. **ماذا يعني «تدريب» هذا الوكيل** — ثلاث آليات مختلفة تمامًا
2. **القاعدة الذهبية** — لا تُغذِّ الوكيل بكود لم يُراجَع
3. **المسار العملي** — من توليد الكود إلى معرفة دائمة
4. **الدروس** — قلب التعلّم، ومتى تُسجَّل
5. **بطاقات المعرفة** — ترقية الدرس إلى قاعدة أو نمط
6. **اقتصاد الـ tokens** — بالأرقام المقيسة على مشاريعك
7. **دورة العمل اليومية** — جلسة نموذجية كاملة
8. **قياس التقدّم** — كيف تعرف أن الوكيل يتحسّن
9. **الطريق إلى نموذج محلي** — المرحلتان الرابعة والخامسة
10. **قوائم مرجعية**

1. ماذا يعني «تدريب» هذا الوكيل

كلمة «تدريب» تُستعمل عادةً بمعنى تعديل أوزان نموذج. هنا تعني شيئًا آخر، وأرخص، وأكثر فائدة في حالتك.

هناك ثلاث آليات مختلفة تمامًا، والخلط بينها هو أكثر ما يضيّع الوقت:

الآلية	ما تفعله	هل تخفّض الاستهلاك؟	الحكم
فهرسة الكود المعتمَد	كل ملف Dart تقبله ويدخل مشروعك يصبح قابلًا للاسترجاع	نعم — يغني Claude عن قراءة الملفات	**تلقائي**، يكفي `flutter-agent index`
تسجيل الدروس	كل تصحيح تكتبه يُحفظ ويُستدعى لاحقًا	غير مباشر — يمنع إعادة العمل، وهي أغلى تكلفة	**افعله.** هذا جوهر الدليل
تدريب نموذج محلي (fine-tuning)	تعديل أوزان نموذج صغير على أسلوبك	**لا** — إلا إن نقلتَ العمل فعليًا بعيدًا عن Claude	مؤجَّل، راجع القسم 9

الفرق الجوهري: الاسترجاع مقابل التدريب

التدريب يُرسّخ الأسلوب في أوزان النموذج. مكلف، بطيء، غير قابل للتراجع، ولا يمنع النموذج من اختراع دوال غير موجودة.

الاسترجاع يضع المثال الصحيح أمام النموذج لحظة الحاجة. فوري، قابل للقراءة والتعديل والحذف، ولا يخترع شيئًا لأنه يعرض كودًا حقيقيًا من مشروعك.

> في حالتك — كرت شاشة بسعة 6 غيغابايت وقرابة 900 ملف Dart — الاسترجاع يعطيك معظم الفائدة بجزء يسير من التكلفة. لذلك هذا الدليل كلّه عن الآليتين الأولى والثانية، والثالثة مؤجّلة بوعي.

ما الذي «يتعلّمه» الوكيل فعلًا

- **كيف تكتب أنت** — من الكود المفهرس: أسماؤك، بنيتك، طريقتك في معالجة الأخطاء.
- **ما الذي أخطأ فيه Claude سابقًا** — من الدروس.
- **لماذا القاعدة قاعدة** — من بطاقات `rules/`.
- **كيف يبدو الحل النموذجي** — من بطاقات `patterns/`.

2. القاعدة الذهبية

لا تُغذِّ الوكيل بكود ولّده Claude ولم تراجعه. هذا أسرع طريق لإفساده.

الحلقة الخطرة

تخيّل هذا المسار:

```
Claude writes a cubit  ->  it looks fine  ->  indexed as a "golden example"
        ^                                                  |
        |                                                  v
   next session retrieves it as the precedent  <--------------+
```

إن كان في ذلك الـ cubit خطأ صغير — نسيان `if (isClosed) return;` مثلًا — فإنه لا يبقى خطأً واحدًا، بل يصير *السابقة* التي يُحتذى بها في كل جلسة لاحقة. الخطأ يتضاعف بدل أن يُصحَّح.

هذا ما يُعرف في الأدبيات بانهيار النموذج (model collapse) مصغَّرًا: نظام يتغذّى على مخرجاته فيبتعد تدريجيًا عن الواقع.

المرشِّح الصحيح

مرشِّح خاطئ	مرشِّح صحيح
«هل كتبه Claude؟»	«هل اجتاز `flutter analyze` ؟»
«هل يبدو جيدًا؟»	«هل اجتاز `flutter-agent check` ؟»
«هل يعمل؟»	«هل راجعتَه أنت وقبلته ودخل المشروع؟»

الخلاصة: الكود لا يدخل قاعدة المعرفة لأن Claude كتبه، بل لأنه **شُحن**. الشحن هو شهادة الجودة الوحيدة الموثوقة، لأنه يعني أن إنسانًا قرأه وقَبِله.

ولا تحتاج إلى فعل شيء

هذا المرشِّح مطبَّق أصلًا بحكم التصميم: الوكيل يفهرس **مجلدات مشاريعك المسجّلة**، لا ما يظهر في نافذة المحادثة. الكود الذي ترفضه لا يصل إلى القرص أصلًا، فلا يُفهرس. وكل ما عليك هو ألّا تحفظ كودًا لم تراجعه.

3. المسار العملي

من طلب تكتبه إلى معرفة دائمة داخل الوكيل، في ست خطوات.

الخطوة 1 — اسأل الوكيل قبل أن تطلب الكود

ابدأ الجلسة بإعطاء Claude السابقة، لا بوصف المطلوب فقط. قل مثلًا:

> «قبل أن تكتب شيئًا، استخدم `flutter_pattern` لمعرفة كيف نتعامل مع الأخطاء، ثم `flutter_component` لتعرف ما المكوّنات الجاهزة.»

هذا وحده يرفع جودة الناتج بشكل ملموس، ويخفّض الاستهلاك لأن Claude لن يقرأ ملفات كاملة ليستنتج أسلوبك.

الخطوة 2 — ولّد الهيكل بالوكيل لا بـ Claude

```
flutter-agent scaffold notifications --root "C:\...\bilhikma" --write
```

ثمانية ملفات تُولَّد محليًا بتكلفة صفر من الـ tokens. دع Claude يكتب الجزء الذي يحتاج تفكيرًا فعلًا: جسم الـ data source والمنطق الخاص بالـ cubit.

الخطوة 3 — بوّابتا التحقق

```
cd "C:\Users\IRD-83\Desktop\my apps\bilhikma"
flutter analyze lib/features/notifications
flutter-agent check lib/features/notifications
```

البوابة	ما تضمنه
`flutter analyze`	أن الكود يُصرَّف ويخلو من أخطاء المحلّل
`flutter-agent check`	أن الكود يحترم معماريتك: الطبقات، الترجمة، أحجام الملفات

البوابة الأولى تقول «يعمل»، والثانية تقول «يعمل *بأسلوبنا*». الكود لا يُقبل إلا باجتياز الاثنتين.

الخطوة 4 — راجع أنت

الأداتان لا تحكمان على صحة المنطق. اقرأ الكود. هذه الخطوة غير قابلة للأتمتة، وهي سبب جودة قاعدة معرفتك.

الخطوة 5 — سجّل ما تعلّمته من التصحيح

إن اضطررت لتصحيح شيء، فذلك التصحيح هو الناتج الأثمن من الجلسة كلها. سجّله فورًا:

> قل لـ Claude: «احفظ هذا كدرس» — فيستدعي `flutter_lesson_add` بالتفاصيل الصحيحة دون أن تكتبها بنفسك.

الخطوة 6 — أعد الفهرسة

```
flutter-agent index
```

الفهرسة تراكمية، فلن يُعاد حساب إلا ما تغيّر — عادةً أقل من ثانيتين. الآن صار الكود الجديد والدرس الجديد جزءًا من ذاكرة الوكيل.

المسار كاملًا

```
ask flutter_pattern  ->  scaffold locally  ->  Claude writes the thinking part
        |
        v
flutter analyze  ->  flutter-agent check  ->  YOUR review
        |                                          |
        | (failed)                                 | (accepted)
        v                                          v
  record a lesson  --------------------------->  flutter-agent index
```

4. الدروس — قلب التعلّم

الدرس تصحيح دائم: ما كُتب، وما كان يجب أن يُكتب، ولماذا. وهو الآلية الوحيدة التي تجعل خطأ اليوم مستحيلًا غدًا.

متى تسجّل درسًا

#	الحالة	ماذا تسجّل
1	صحّحتَ كود Flutter الذي كتبه Claude	النسخة الخاطئة والصحيحة والسبب
2	`flutter analyze` رفض كودًا مولّدًا	نصّ الخطأ وإصلاحه
3	`flutter-agent check` وجد مخالفة ثم أُصلحت	المخالفة والإصلاح مع `--rule`
4	تبيّن عُرف لا تغطّيه أي قاعدة	العُرف؛ وإن تكرّر فارفعه إلى بطاقة

> **لا تسجّل درسًا لكل ملاحظة عابرة.** الدرس ليس مفكرة. إن لم يكن التصحيح قابلًا لأن يتكرر في مشروع آخر بعد ثلاثة أشهر، فلا تسجّله — ستغرق قاعدة المعرفة في ضجيج يُضعف الاسترجاع.

تشريح درس جيّد مقابل درس سيّئ

درس سيّئ

```
topic: "cubit"
right: "use the repo properly"
why:   "because it is better"
```

لماذا هو سيّئ: لا كود، ولا سياق، ولا قرار قابل للتطبيق. حين يسترجعه Claude لن يستفيد منه شيئًا، وسيكون قد استهلك مساحة في النتيجة بلا مقابل.

درس جيّد

```
topic: "cubit registered as singleton"

wrong: getIt..registerLazySingleton(() => ArchiveCubit(repo: getIt()));

right: getIt..registerFactory(() => ArchiveCubit(repo: getIt()));

why:   A singleton cubit keeps the previous screen's state and emits
       after close(). Cubits are always registerFactory.

tags:  di, cubit        layer: cubit        rule: R6
```

لماذا هو جيّد: فيه السطر الخاطئ والسطر الصحيح معًا، وسببٌ يشرح *النتيجة* لا القاعدة، ووسوم تجعله قابلًا للاسترجاع من اتجاهات عدة.

لماذا حقل `wrong` مهم بقدر `right`

الاسترجاع يعمل بتقارب المعنى. حين يكون Claude على وشك كتابة `registerLazySingleton` لـ cubit، فإن النص الأقرب لما في ذهنه هو **النسخة الخاطئة** لا الصحيحة. وجود الخطأ مكتوبًا هو ما يجعل الدرس يظهر في اللحظة المناسبة.

وهو كذلك ما يحوّل الدرس لاحقًا إلى زوج تدريب صالح (سؤال/جواب) عند التصدير.

الحقول

`topic`	**إلزامي.** وصف قصير دقيق — هو ما يُطابَق عند البحث
`right`	**إلزامي.** الصيغة الصحيحة، ككود حقيقي كلما أمكن
`wrong`	ما كُتب فعلًا بدلًا منها
`why`	النتيجة العملية للخطأ، لا إعادة صياغة القاعدة
`tags`	كلمات تسهّل الاسترجاع: di، cubit، errors، pagination
`layer`	الطبقة: cubit، data_source، screen، widget...
`feature / project`	السياق، إن كان الدرس خاصًا بموضع بعينه
`rule`	رمز القاعدة المرتبطة (R1...R12) ليظهر الدرس بجوارها

التسجيل عمليًا

الطريقة المفضَّلة: اطلب من Claude

بعد أن تصحّح له شيئًا، قل ببساطة: **«احفظ هذا كدرس»**. سيملأ الحقول من سياق المحادثة — وهو يعرف ما كتبه وما صحّحته أنت — ويستدعي الأداة.

الطريقة اليدوية

```
flutter-agent lesson "cubit registered as singleton" `
  "getIt..registerFactory(() => ArchiveCubit(repo: getIt()));" `
  --wrong "getIt..registerLazySingleton(() => ArchiveCubit(repo: getIt()));" `
  --why "A singleton cubit keeps the previous screen's state and emits after close()." `
  --tag di --tag cubit --layer cubit --rule R6
```

الدرس يُفهرس تلقائيًا بعد التسجيل، فيصبح قابلًا للاسترجاع في الجلسة نفسها.

الاسترجاع والمراجعة والحذف

```
flutter-agent lessons                              # the most recent
flutter-agent lessons "how do I register a cubit"  # by relevance
```

الدروس تظهر تلقائيًا أيضًا ضمن نتائج `flutter-agent pattern` ، وهذا هو المقصود: أن يصل التصحيح قبل تكرار الخطأ لا بعده.

والملف `knowledge/lessons/lessons.jsonl` نصّي عادي: افتحه، احذف سطرًا تبيّن خطؤه، ثم أعد الفهرسة. **درس خاطئ أسوأ من لا درس**، لأنه يُرسّخ الخطأ بثقة.

5. بطاقات المعرفة

حين يتكرّر الدرس نفسه، فالمشكلة ليست في جلسة بعينها بل في غياب قاعدة. عندها تُرفَع إلى بطاقة.

الفرق بين الدرس والبطاقة

	الدرس	البطاقة
الحجم	تصحيح واحد	موضوع كامل
المصدر	حادثة في جلسة	نمط متكرر في الكود
الصيغة	سطر في JSONL	ملف Markdown
الكتابة	تلقائية أثناء العمل	متعمَّدة، مرة واحدة

النوعان

المجلد	يجيب عن	الموجود حاليًا
`knowledge/rules/`	**لماذا** هذا القيد موجود، مع مثال قبل/بعد	layering، localization، errors، file-size
`knowledge/patterns/`	**كيف** نكتب هذا عندنا، بمثال نموذجي	cubit-and-state، screen-and-body، data-source-and-model، pagination، dependency-injection

أسرع طريقة لكتابة بطاقة

لا تكتبها من ذاكرتك — اطلب من Claude أن يستخرجها من كودك الحقيقي:

> «اقرأ كيف ننفّذ رفع الصور في `skygate` و `hikmaapp` ، ثم اكتب بطاقة نمط في `knowledge/patterns/image-upload.md` على نسق البطاقات الموجودة: كودٌ حقيقي من المشروع، وشرحٌ للقرارات لا للصياغة، وتضمين النسخة الخاطئة الشائعة.»

ثم راجعها وأعد الفهرسة. هذه أعلى عمليات التدريب مردودًا: بطاقة واحدة جيدة تخدم كل جلسة لاحقة تمسّ الموضوع.

معايير البطاقة الجيدة

- **كود حقيقي من مشاريعك**، لا أمثلة مخترعة. المثال المخترع يعلّم Claude أسلوبًا لا تستعمله.
- **اشرح القرار لا الصياغة.** «لماذا وُجد `clearError` » مفيد؛ « `copyWith` ينسخ الحالة» ليس كذلك.
- **أدرج النسخة الخاطئة** للسبب نفسه المذكور في القسم السابق.
- **موضوع واحد لكل ملف**، بعناوين `##` — فالمقطِّع يعتمد عليها في التقسيم.
- **اربطها بالقواعد** بذكر رموزها (R1…R12) في نهاية البطاقة.

المواضيع التي تستحق بطاقة عندك

بناءً على ما هو موجود في كودك وما لا تغطّيه البطاقات الحالية:

الموضوع	المصدر في مشاريعك
رفع الصور وتحديث الملف الشخصي	skygate، hikmaapp
المفضّلة والحفظ المحلي	bilhikma / saved lessons
التحقق من النماذج (forms) والـ validation	auth في bilhikma وskygate
التوجيه والتنقّل (routing)	core/routing في كل المشاريع
تشغيل الوسائط	videoplayerservice في bilhikma
الصلاحيات والحضور	attendence

6. اقتصاد الـ tokens

كل الأرقام أدناه مقيسة فعليًا على مشروع bilhikma، لا تقديرات.

الأثر المباشر

المهمة	بدون الوكيل	مع الوكيل	التوفير
مراجعة feature	قراءة 8 ملفات ≈ 11,793 حرفًا	92 ← `check` حرفًا	**99٪**
«كيف نفعل كذا عندنا؟»	feature + core ≈ 27,900 حرفًا	1,231 ← `pattern` حرفًا	**96٪**
«أي widget جاهز؟»	قراءة الحزمة كاملة	689 ← `component` حرفًا	—
إنشاء feature	كتابة 8 ملفات ≈ 12,000 token مُخرَج	صفر ← `scaffold`	**~100٪**

> **السطر الأخير هو الأهم.** الـ tokens المُخرَجة أغلى من المُدخَلة، وتوليد الهيكل محليًا يلغيها بالكامل. اجعل `scaffold` عادة قبل أن تطلب من Claude كتابة ملفات جديدة.

التكلفة الثابتة — بصراحة

خادم MCP يضيف نحو 1,200 token من تعريفات الأدوات إلى **كل** جلسة، سواء استُخدمت الأدوات أم لا. وهي تكلفة تُستردّ أول مرة يُغني فيها الوكيل عن قراءة ملف واحد — لكنها ليست صفرًا، ومن الأمانة ذكرها.

ملاحظة على الإعدادات الافتراضية

كانت `pattern` تعيد 7,823 حرفًا، أي ما يقارب قراءة الملفات نفسها — وهو ما يُبطل الغاية. بعد ضبط الإعدادات صارت تعيد **1,231 حرفًا**، أي أقل بـ 6.4 مرة، مع احتفاظها بالقاعدة والمثال والدروس.

إن أردت نتائج أوسع عند الحاجة، ارفع العدد صراحةً:

```
flutter-agent pattern "pagination" -n 3
flutter-agent search "error mapping" -n 5
```

عادات تخفّض الاستهلاك

افعل	لا تفعل
ابدأ بـ `pattern` أو `component`	لا تطلب من Claude قراءة مجلد كامل ليفهم الأسلوب
ولّد الهيكل بـ `scaffold`	لا تطلب كتابة ثمانية ملفات يدويًا
راجع بـ `check` على مجلد feature واحد	لا تُشغّل `check` على `lib/` إلا لتدقيق الترجمة
استخدم `--paths-only` حين تريد المواضع فقط	لا تجلب نص الكود إن كنت ستفتح الملف أصلًا
سجّل الدروس لتمنع إعادة العمل	لا تكرّر شرح العُرف نفسه في كل جلسة

وأغلى ما يُستهلك ليس القراءة ولا الكتابة، بل **إعادة العمل**: كود خاطئ، ثم تصحيح، ثم إعادة توليد. الدروس تهاجم هذا البند تحديدًا.

رافعة إضافية: ملف CLAUDE.md

ملفك العام `~/.claude/CLAUDE.md` يبلغ 6,512 حرفًا (نحو 1,700 token) ويُحمَّل في **كل جلسة من كل مشروع**. ومعظم محتواه صار متاحًا عند الطلب عبر `flutter_pattern` .

اختصاره إلى إشارة قصيرة توجّه Claude إلى الوكيل يوفّر قرابة 1,300 token في كل جلسة بشكل دائم. وهو تغيير في إعدادك العام، فاتخذه بوعي.

7. دورة العمل اليومية

جلسة نموذجية كاملة، بما تقوله أنت وما يجري خلفه.

أنت

> «أريد feature للإشعارات في bilhikma. ابدأ بـ `flutter_pattern` لمعرفة أسلوبنا في معالجة الأخطاء والقوائم، و `flutter_component` لتعرف المكوّنات الجاهزة. لا تكتب كودًا قبل ذلك.»

Claude

يستدعي الأداتين (نحو 1,900 حرف إجمالًا)، ثم يقترح الخطة بناءً على كودك الحقيقي: `AppEmptyState` للحالة الفارغة، وعقد الأخطاء القائم على مفاتيح قابلة للترجمة.

أنت

> «جيد. ولّد الهيكل بـ `flutter_scaffold` ، ثم اكتب فقط جسم الـ data source مقابل نقطة النهاية `/user/notifications`.»

Claude

يولّد الثمانية ملفات محليًا بتكلفة صفر، ويكتب جسم دالة واحدة. ثم يشغّل `flutter analyze` و `flutter_check` .

أنت

تقرأ الكود وتلاحظ أنه سجّل الـ cubit كـ `registerLazySingleton` . تصحّحه وتقول:

> «الـ cubits دائمًا `registerFactory` عندنا. احفظ هذا كدرس.»

Claude

يستدعي `flutter_lesson_add` بالحقول الصحيحة، فيُفهرس الدرس فورًا.

النتيجة

feature يعمل، وقاعدة معرفة أغنى بدرس واحد، وجلسة استهلكت جزءًا يسيرًا مما كانت ستستهلكه. وفي المرة القادمة التي يقترب فيها Claude من تسجيل cubit، سيصل التصحيح قبل الخطأ.

عبارات جاهزة

الغرض	ما تقوله
قبل التنفيذ	«استخدم `flutter_pattern` أولًا ولا تكتب قبل ذلك»
قبل كتابة widget	«تحقّق من `flutter_component` قبل أن تنشئ مكوّنًا جديدًا»
للمراجعة	«راجع هذا المجلد بـ `flutter_check` »
بعد التصحيح	«احفظ هذا كدرس»
لبناء بطاقة	«اكتب بطاقة نمط لـ كذا من كودنا في المشروع الفلاني»
لتقسيم ملف	«استخدم `flutter_refactor_plan` ثم نفّذ الخطة»

8. قياس التقدّم

«هل يتحسّن الوكيل فعلًا؟» سؤال يُجاب بالأرقام لا بالانطباع.

المؤشرات

```
flutter-agent status
```

راقب سطر `kinds` تحديدًا. الوضع الحالي:

```
kinds:   code=1617, pattern=27, rule=19, lesson=1
```

المؤشر	اليوم	هدف معقول خلال شهرين	دلالته
`lesson`	1	30 – 60	كم تصحيحًا صار دائمًا
`pattern`	27	60 – 90	تغطية المواضيع المتكررة
`rule`	19	30 – 40	عمق تفسير القيود
`code`	1617	ينمو مع مشاريعك	حجم السابقة المتاحة

> **المؤشر الأهم هو `lesson`.** بقاؤه عند 1 بعد شهر من العمل لا يعني أن Claude لم يخطئ، بل يعني أنك لم تسجّل. كل تصحيح غير مسجَّل ستدفع ثمنه مرة أخرى.

اختبار الجودة العملي

احتفظ بقائمة ثابتة من الأسئلة التي تعرف إجاباتها، وأعد تشغيلها بعد كل إضافة كبيرة لقاعدة المعرفة:

```
flutter-agent pattern "error handling"
flutter-agent pattern "pagination"
flutter-agent search "empty state widget"
flutter-agent component AppButton
```

إن عادت النتائج بالملفات التي تتوقعها، فالاسترجاع سليم. وإن عادت ببطاقة لا علاقة لها بالسؤال — كما يحدث اليوم مع موضوع رفع الصور لغياب بطاقته — فتلك إشارة إلى ثغرة تستحق بطاقة جديدة.

صحة الفهرس

```
flutter-agent status      # health: ok
flutter-agent doctor      # device: cuda
pytest tests -q           # 53 passed
```

إن ظهر `health: BROKEN` فقد اختلف FAISS مع SQLite؛ العلاج `flutter-agent index --force` .

9. الطريق إلى نموذج محلي

هذا هو المعنى التقليدي لكلمة «تدريب». وهو مؤجَّل بوعي، لأسباب تستحق الشرح.

المرحلة الرابعة: نموذج محلي للأعمال الروتينية

تثبيت Ollama مع `qwen2.5-coder:7b-instruct-q4_K_M` (نحو 4.5 غيغابايت) ليتولّى الأعمال الميكانيكية: ملء model من عيّنة JSON، كتابة class للحالة، صياغة widget بسيط — مع حقن السياق المسترجَع دائمًا، وبوابة `flutter analyze` دائمًا.

هذه المرحلة وحدها هي ما يخفّض استهلاك Claude في خطوة التوليد نفسها، لأن العمل ينتقل فعليًا إلى جهازك.

> **قيد عملي:** كرت RTX 4050 لديك سعته 6.1 غيغابايت، وكان مشغولًا بنحو 5.0 منها (ComfyUI غالبًا) وقت القياس. يلزم تحرير تلك المساحة قبل تحميل نموذج 7B.

المرحلة الخامسة: QLoRA

الدروس المسجَّلة قابلة للتصدير مباشرةً كبيانات تدريب:

```
flutter-agent export-dataset lessons.jsonl
```

تُنتج ثلاثيات `assistant` / `user` / `system` جاهزة لأي سكربت تدريب.

متى يستحق ذلك؟

الشرط	الحالة
عدد دروس كافٍ (بضع مئات)	لديك 1 اليوم
ذاكرة GPU متاحة	6.1 غيغابايت، وهي على الحد
عائد واضح مقابل الاسترجاع	غير مثبت بعد

الرأي الصريح

على كرت محمول بسعة 6 غيغابايت، تدريب QLoRA على ملفات Dart الطويلة بطيء، وعائده متواضع: التدريب يرسّخ **الأسلوب** لا **الصحة**، وسيظل النموذج يخترع دوال غير موجودة. أما الاسترجاع مع محرّك القواعد فيعطيك معظم الفائدة بتكلفة أقل كثيرًا وبقابلية كاملة للمراجعة والتراجع.

الأصل الحقيقي هو البيانات لا النموذج. اجمع الدروس بانتظام، وسيبقى قرار التدريب متاحًا متى نضجت الظروف — بينما قاعدة المعرفة تفيدك من اليوم الأول.

10. قوائم مرجعية

قبل قبول كود ولّده Claude

- اجتاز `flutter analyze` بلا أخطاء
- اجتاز `flutter-agent check` على مجلد الـ feature
- لا يوجد استيراد `package:<project>/…` داخل `lib/`
- لا نصوص واجهة مكتوبة مباشرة؛ كل المفاتيح في LangKeys وفي ملفي الترجمة
- الـ cubit خالٍ من `BuildContext`، وفيه `if (isClosed) return;` بعد كل `await`
- الـ cubit مسجَّل بـ `registerFactory` لا `registerLazySingleton`
- قرأتَه أنت وفهمت منطقه

قبل تسجيل درس

- هل سيتكرر هذا الخطأ في مشروع آخر؟ إن لا، فلا تسجّله
- هل حقل `right` كودٌ حقيقي لا نصيحة عامة؟
- هل حقل `wrong` مملوء؟ (هو ما يجعل الدرس يُسترجَع في وقته)
- هل `why` يشرح النتيجة لا القاعدة؟
- هل أضفت `--layer` و`--rule` متى انطبقا؟

مراجعة أسبوعية (عشر دقائق)

- `flutter-agent status` — هل ارتفع عدد الدروس هذا الأسبوع؟
- `flutter-agent lessons` — اقرأ الجديد، واحذف ما تبيّن خطؤه
- هل تكرّر درسان في الموضوع نفسه؟ ارفعهما إلى بطاقة
- شغّل أسئلة الجودة الثابتة وتحقّق من النتائج
- `flutter-agent index` بعد أي تعديل على `knowledge/`

الملخّص في أربعة أسطر

1. اسأل الوكيل قبل أن تطلب الكود.
2. ولّد الهيكل محليًا، ودع Claude للجزء الذي يحتاج تفكيرًا.
3. لا تقبل كودًا لم يجتز البوابتين ومراجعتك.
4. سجّل كل تصحيح — فهو الفرق بين وكيل يتحسّن وآخر يراوح مكانه.
